

SSD_GPIO usage reference

1 Overview

1.1. Overview

GPIO adopts a standard LINUX framework and can use a unified interface to operate gpio.

1.2. GPIO NUM and PAD correspondence table

Please check the PAD name of the GPIO on the hardware circuit diagram, find this table, and the corresponding num is the num of the GPIO you want to operate.

For example: a GPIO on the hardware circuit diagram is PAD_FUART_RX, if you want to operate this GPIO, the corresponding num=14.

List of open GPIOs:

Table 1-1

PAD_GPIO0	0	PAD_TTL9	28	PAD_SD_D3
PAD_GPIO0	0	PAD_TTL9	28	PAD_SD_D3

PAD_GPIO1	1	PAD_TTL10	29	PAD_SD_GPIO
PAD_GPIO2	2	PAD_TTL11	30	PAD_PM_SD_CDZ
PAD_GPIO3	3	PAD_TTL12	31	PAD_PM_IRIN
PAD_GPIO4	4	PAD_TTL13	32	PADA_IDAC_OUT_B
PAD_GPIO5	5	PAD_TTL14	33	PADA_IDAC_OUT_G
PAD_GPIO6	6	PAD_TTL15	34	PADA_IDAC_OUT_R
PAD_GPIO7	7	PAD_TTL16	35	PAD_PM_SPI_CZ
PAD_GPIO8	8	PAD_TTL17	36	PAD_PM_SPI_CK
PAD_GPIO9	9	PAD_TTL18	37	PAD_PM_SPI_DI
PAD_GPIO12	10	PAD_TTL19	38	PAD_PM_SPI_DO
PAD_GPIO13	11	PAD_TTL20	39	PAD_PM_SPI_WPZ
PAD_GPIO14	12	PAD_TTL21	40	PAD_PM_SPI_HLD
PAD_GPIO15	13	PAD_TTL22	41	PAD_PM_LED0
PAD_FUART_RX	14	PAD_TTL23	42	PAD_PM_LED1
PAD_FUART_RX	15	PAD_TTL24	43	PAD_SAR_GPIO0
PAD_FUART_TX	16	PAD_TTL25	44	PAD_SAR_GPIO1
PAD_FUART_CTS	17	PAD_TTL26	45	PAD_SAR_GPIO2
PAD_FUART_RTS	18	PAD_TTL27	46	PAD_SAR_GPIO3
PAD_TTL0	19	PAD_UART0_RX	47	PAD_ETH_RN
PAD_TTL1	20	PAD_UART0_TX	48	PAD_ETH_RP
PAD_TTL2	twenty one	PAD_UART1_RX	49	PAD_ETH_TN
PAD_GPIO0	0	PAD_TTL9	28	PAD_SD_D3

PAD_TTL3	22	PAD_UART1_TX	50	PAD_ETH_TP
PAD_TTL4	23	PAD_SD_CLK	51	PAD_DM_P1
PAD_TTL5	24	PAD_SD_CMD	52	PAD_DP_P1
PAD_TTL6	25	PAD_SD_D0	53	PAD_DM_P2
PAD_TTL7	26	PAD_SD_D1	54	PAD_DP_P2
PAD_TTL8	27	PAD_SD_D2	55	PAD_DM_P3



2. 内核使用GPIO

2.1. 申请为gpio端口

- 目的
创建端口为GPIO。
- 语法

```
int gpio_request(unsigned gpio, const char *label)
```

- 参数

参数名称	描述
gpio	Gpio num
label	具体名称

- 返回值

返回值	描述
0	成功。
Other	失败。

2.2. 设为输入

- 目的
标记gpio为输入。
- 语法

```
int gpio_direction_input(unsigned gpio);
```

- 参数

参数名称	描述
gpio	Gpio num

- 返回值

返回值	描述
0	成功。
Other	失败。

2.3. 设为输出

- 目的
标记gpio为输出。

- 语法

```
int gpio_direction_output(unsigned gpio, int value);
```

- 参数

参数名称	描述
gpio	Gpio num
value	输出值

- 返回值

返回值	描述
0	成功。
Other	失败。

2.4. 获取输入电平

- 目的

获取输入引脚的电平。

- 语法

```
int gpio_get_value(unsigned gpio);
```

- 参数

参数名称	描述
gpio	Gpio num

- 返回值

返回值	描述
Int	电平值

2.5. 设置输出电平

- 目的

设定输出引脚的电平。

- 语法

```
void gpio_set_value(unsigned gpio, int value);
```

- 参数

参数名称	描述
gpio	Gpio num
value	输出值

- 返回值

返回值	描述
0	成功。
Other	失败。

3. 用户空间使用GPIO

用户空间访问gpio，即通过sysfs接口访问gpio

下面是/sys/class/gpio目录下的三种文件：

--export/unexport文件

--gpioN指代具体的gpio引脚

--gpio_chipN指代gpio控制器

必须知道以上接口没有标准device文件和它们的链接。

```
/sys/class/gpio # ls
export
gpiochip0
unexport
/sys/class/gpio #
```

图3-1

3.1. export/unexport文件接口

`/sys/class/gpio/export`，该接口只能写不能读。

用户程序通过写入gpio的编号来向内核申请将某个gpio的控制权导出到用户空间，前提是没有内核代码申请这个gpio端口，如用户申请编号为12的GPIO的命令：

```
echo 12 > export
```

上述操作会为12号gpio创建一个节点gpio12，此时/sys/class/gpio目录下边生成一个gpio12的目录，如下图所示：

```
/sys/class/gpio #
/sys/class/gpio # echo 12 > export
[ 7067.555358] [GPIO][00134] [mstar-gpio]mstar_gpio_request offset=12
[ 7067.561649] [GPIO][00180] [mstar-gpio]mstar_gpio_to_irq,but not set reg
/sys/class/gpio # ls
export
gpio12
gpiochip0
unexport
/sys/class/gpio #
/sys/class/gpio #
```

图3-2

/sys/class/gpio/unexport和导出的效果相反，比如移除gpio12这个节点操作命令：

```
echo 12 > unexport
```

上述操作将会移除gpio12这个节点，如下图所示：

```
/sys/class/gpio # echo 12 > unexport
[ 7246.139183] [GPIO][00140] [mstar-gpio]mstar_gpio_free
/sys/class/gpio #
/sys/class/gpio #
/sys/class/gpio #
/sys/class/gpio # ls
export
gpiochip0
unexport
/sys/class/gpio #
/sys/class/gpio #
```

图3-3

3.2. /sys/class/gpio/gpioN

指代某个具体的gpio端口,里边有如下属性文件：

direction 表示gpio端口的方向，读取结果是in或out。也可以对该文件进行写操作，写入out时该gpio设为输出同时电平默认为低。写入low或high时不仅可以设置为输出还可以设置指定的输出电平。当然如果内核不支持或者内核代码不愿意，将不会存在这个属性，比如内核调用了gpio_export(N,0)就表示内核不愿意修改gpio端口方向属性。

value 表示gpio引脚的电平，0表示低电平，1表示高电平；如果gpio被配置为输出，这个值是可写的，记住任何非零的值都将输出为高电平。如果某个引脚被配置为中断，则可以调用poll(2)函数监听该中断，中断触发后poll(2)函数就会返回。


```

/sys/class/gpio # echo 12 > export
[ 7446.305126] [GPIO][00134] [mstar-gpio]mstar_gpio_request offset=12
[ 7446.311406] [GPIO][00180] [mstar-gpio]mstar_gpio_to_irq,but not set reg
/sys/class/gpio #
/sys/class/gpio # ls
export
gpio12
gpiochip0
unexport
/sys/class/gpio # cd gpio12/
/sys/devices/virtual/gpio/gpio12 # ls
active_low
direction
edge
power
subsystem
uevent
value
/sys/devices/virtual/gpio/gpio12 # echo out > direction
[ 7496.197243] [GPIO][00173] [mstar-gpio]mstar_gpio_direction_output
/sys/devices/virtual/gpio/gpio12 # echo 1 > value
[ 7512.375372] [GPIO][00149] [mstar-gpio]mstar_gpio_set
/sys/devices/virtual/gpio/gpio12 # echo 0 > value
[ 7518.275654] [GPIO][00]mstar_gpio_set

/sys/devices/virtual/gpio/gpio12 # echo 1 > value
[ 7526.954376] [GPIO][00149] [mstar-gpio]mstar_gpio_set
/sys/devices/virtual/gpio/gpio12 #

```

图3-4

3.3. 示例

非中断模式

提供用户空间通过sysfs接口操作gpio的sample code源文件：[gpio_main.c](#)

[media/gpio_main.c]

写操作：int ss_gpio_set_value(unsigned int gpio, unsigned int value) 传入要写的gpio的编号，输入参数1/0，控制高/低电平；

读操作：int ss_gpio_get_value(unsigned int gpio, unsigned int *value) 传入要写的gpio的编号，获取当前电平状态。

编译sample code生成gpio_main可执行bin档：

```
# arm-linux-gnueabi-gcc -o gpio_main gpio_main.c
```

在shell中运行下述命令测试gpio：

```
# ./gpio_main input
# ./gpio_main output 1
```

提供用户空间通过mmap直接访问寄存器操作gpio的sample code源文件：
[gpio_mmap.c](#) [media/gpio_mmap.c]

使用此方式操作gpio，用户需要查看GPIO_Mapping_Table，确认gpio的bank地址以及偏移地址。以PAD_GPIO0为例：

Ball Name	tion(defa	Configuration(register settings)	OEN 0: output / 1: in	IN	OUT
PAD_GPIO0		reg_all_pad_in; reg[101EA1]#7=0breg_spi0_	reg[103C00]#5	reg[103C00]#0	reg[103C00]#4
PAD_GPIO1		reg_all_pad_in; reg[101EA1]#7=0b reg_spi0_mode; reg[101E18]#2 ~ #0=0b reg_fuart_mode; reg[101E06]#2 ~ #0=0b reg_pwm1_mode; reg[101E0E]#5 ~ #3=0b reg_eth1_mode; reg[101E1D]#3 ~ #0=0b reg_i2s_mode; reg[101E1F]#5 ~ #4=0b	reg[103C02]#5	reg[103C02]#0	reg[103C02]#4

如上表看PAD_GPIO1的话，bank基地址为0x103C，偏移地址为
(0x02/2=0x01)，BIT5配置gpio方向，BIT4控制输出电平，BIT0的值为输入电
平状态。

4. UBOOT使用GPIO

4.1. CMD：gpio -Config gpio port

```
Usage:

gpio (for 2nd parameter, you must type at least 3 characters)

gpio output <gpio#> <1/0> : ex: gpio output 69 1

gpio input/get <gpio#> : ex: gpio input 10 (gpio 10 set as
input)

gpio toggle <gpio#> : ex: gpio tog 49 (toggle)
```

```
gpio state <gpio#>          : ex: gpio sta 49 (get i/o
status(direction) & pin status)

gpio list [num_of_pins]      : ex: gpio list 10 (list GPI01~GPI010
status)
```

4.2. API

4.2.1. 设为输入

- 目的
标记gpio为输入。
- 语法

```
void MDrv_GPIO_Pad_Odn(MS_GPIO_NUM u32IndexGPIO);
```

- 参数

参数名称	描述
u32IndexGPIO	Gpio num

- 返回值

返回值	描述
void	

4.2.2. 设为输出

- 目的
标记gpio为输出。
- 语法

```
void MDrv_GPIO_Pad_Oen(MS_GPIO_NUM u32IndexGPIO);
```

- 参数

参数名称	描述
u32IndexGPIO	Gpio num

- 返回值

返回值	描述
Void	

4.2.3. 获取输入电平

- 目的

获取输入引脚的电平。

- 语法

```
U8 MDrv_GPIO_Pad_Read(MS_GPIO_NUM u32IndexGPIO);
```

- parameter

parameter name	describe
u32IndexGPIO	Gpio num

- return value

return value	describe
unsigned char	Level value

4.2.4. Set output high level

- Purpose

Set this pin high.

- grammar

```
void MDrv_GPIO_Pull_High(MS_GPIO_NUM u32IndexGPIO);
```

- parameter

parameter name	describe
gpio	Gpio num

4.2.5. Set output low level

- Purpose

Set this pin low.

- grammar

```
void MDrv_GPIO_Pull_Low(MS_GPIO_NUM u32IndexGPIO);
```

- parameter

parameter name	describe
gpio	Gpio num